

Coproduct of Generators, Product of Concepts

Why an *additive* closure code represents a *multiplicative* concept lattice — a lattice-theoretic statement of the factorial-code compression, with a proof and an experiment

Peter Földiák

July 18, 2026

Abstract

Formal Concept Analysis converts a table of yes/no facts into a *concept lattice*. When the attributes split into two independent groups — “size” and “colour,” say — the lattice of the combined data is (up to which combinations actually occur) the *direct product* of the two per-group lattices: one node for every pair, so the node count *multiplies*. That is the blow-up that makes people reach for a factorised representation and ask whether one should learn the factors directly rather than build the product and try to factor it afterwards. This note isolates the clean fact underneath that worry, independently of any particular learning algorithm. We model a concept system as a set of *generators* (a directed acyclic graph of concepts over attribute sinks) decoded by *downward closure*: an object switches on an antichain of generators and inherits the union of the attributes below them. We prove that the *coproduct* (disjoint union) of two such generators over disjoint attribute alphabets generates exactly the *direct product* of the two lattices they generate individually. The generator combines *additively* ($m + n$ nodes) while the represented concept lattice combines *multiplicatively* ($m \cdot n$ nodes). We give the theorem in a decoder-agnostic form (it is really a statement about *separable decoders on disjoint output alphabets*, not about DAGs), work a small example by hand, connect it to Barlow’s factorial codes and to the classical FCA facts about apposition and subdirect products, and confirm it with an experiment in which a description-length learner, handed two independent planted hierarchies, returns the two factors and *zero* cross-factor concepts, representing all their combinations in the object codes. We close with the boundary conditions worth remembering — disjoint alphabets, statistical independence, and realisability — and with why “learn the neural code” and “learn the DAG” turn out not to be a dichotomy.

Contents

1	The phenomenon this note is about	2
2	Generators and the lattice they generate	2
3	Two ways to combine two concept systems	3
4	The result	4
5	What the identity buys, and how it sits in known mathematics	5
6	The neural-code reading	6
7	Boundary conditions worth remembering	7
8	Empirical confirmation	8

1 The phenomenon this note is about

Two independent axes of variation, each with its own little hierarchy, produce a combined concept system whose size is the *product* of the two. If “size” has three meaningful values (small $\subset$ medium $\subset$ large as attribute sets) and “colour” likewise (red, warm, any), then “small red,” “medium warm,” “large any” and every other pairing is a legitimate closed concept of the combined data. Formal Concept Analysis (FCA) [8] will faithfully build all $3 \times 3 = 9$ of them, and with k independent binary axes it will build 2^k . The lattice is correct; it is just *exponentially larger than the description of the world it came from*. The world was two (or k) small independent parts.

The instinct this provokes — and it is the right instinct — is that the meaningful object is not the big product lattice but a *factorisation* of it: keep the parts, and reconstruct any combination on demand. One could build the product and then try to factor it (hard: it is the subdirect decomposition problem, of which more in §5), or one could *generate the factors in the first place* and never form the product at all.

This note is about the second option, and about one fact that makes it work cleanly. Informally:

If you store the two factors side by side (an additive combination of generators) and decode by downward closure, the set of concepts you can represent is already the full product — you get the multiplicative lattice for the additive price, and you never build the product explicitly.

Everything below makes “store side by side,” “decode by closure,” “the lattice you can represent,” and “the full product” precise, proves the identity between them, and marks exactly where it stops holding. The result is stated so that it stands on its own as a fact about closure codes and factorial representations; the description-length learner that motivated it appears only in §8 as a confirmation and in §9 as the application.

2 Generators and the lattice they generate

Fix a finite set of *attributes* M (the observable features; in FCA terms, the columns of a context). We describe a concept system not by listing its concepts but by a small *generator* that produces them.

Definition 1 (Closure DAG). A *closure DAG* G over M is a finite directed acyclic graph whose sinks are the attributes M and whose remaining nodes are *concepts*. For a node k , its *closure* $\text{cl}_G(k) \subseteq M$ is the set of attributes reachable from k (the attributes “below” it). Edges point from a concept to its children (concepts or attributes).

An object does not name a concept lattice; it switches on a few generators and inherits everything below them.

Definition 2 (Antichain code and its decoding). A *code* is an antichain S of nodes of G (no node of S reachable from another). It *decodes* to the attribute set

$$D_G(S) = \bigcup_{k \in S} \text{cl}_G(k) \subseteq M.$$

The *generated family* is $\mathcal{L}(G) = \{D_G(S) : S \text{ an antichain of } G\}$.

The antichain restriction loses nothing at the level of *what can be represented*: closure is monotone along reachability (if u is reachable from v then $\text{cl}_G(u) \subseteq \text{cl}_G(v)$), so in any set of nodes the topmost ones already contribute every attribute the lower ones would. Hence $\mathcal{L}(G) = \{\bigcup_{k \in T} \text{cl}_G(k) : T \subseteq \text{nodes}\}$, the family of attribute sets obtainable as *any* union of node-closures (the antichain is merely the non-redundant way to name each such set).

Lemma 1 ($\mathcal{L}(G)$ is a lattice). $\mathcal{L}(G)$, ordered by $\subseteq$, is a finite lattice. Join is union ($A \vee B = A \cup B$, which lies in $\mathcal{L}(G)$ because the family is closed under union); meet is $A \wedge B = \bigcup\{C \in \mathcal{L}(G) : C \subseteq A \cap B\}$, the largest member below both. Its bottom is $\emptyset$ (the empty union) and its top is $\bigcup_k \text{cl}_G(k)$.

Proof. $\mathcal{L}(G)$ is closed under $\cup$ by construction (a union of two node-unions is a node-union) and contains $\emptyset$. A finite $\cup$ -closed family containing its own empty join is a lattice: the displayed meet is a member (again a union of members), is $\subseteq A \cap B$, and dominates every member below both, so it is the greatest lower bound. $\square$

$\mathcal{L}(G)$ is the concept lattice *our generator commits to*: exactly the attribute sets some object can be predicted to have. It need not be the full FCA lattice of any particular data set — that depends on the data — but it is the right object for asking “what can this generator represent?”.

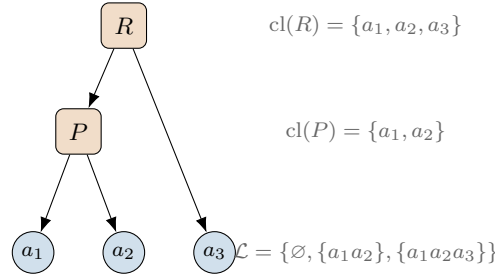


Figure 1: A one-factor closure DAG and the 3-element chain it generates. The two concept nodes $P \subset R$ (with R adding a_3) generate the chain $\emptyset \subset \{a_1a_2\} \subset \{a_1a_2a_3\}$. Two generators, three concepts.

3 Two ways to combine two concept systems

Let G_1 over M_1 and G_2 over M_2 be two closure DAGs. There are two very different things one might mean by “putting them together.”

Definition 3 (Direct product of lattices). The *direct product* $\mathcal{L}(G_1) \times \mathcal{L}(G_2)$ has as elements all ordered pairs (A, B) with $A \in \mathcal{L}(G_1)$, $B \in \mathcal{L}(G_2)$, ordered *componentwise*: $(A, B) \leq (A', B')$ iff $A \subseteq A'$ and $B \subseteq B'$. Meet and join are componentwise. Its size is $|\mathcal{L}(G_1)| \cdot |\mathcal{L}(G_2)|$ — *multiplicative*.

Definition 4 (Coproduct of generators). Assume $M_1 \cap M_2 = \emptyset$. The *coproduct* $G_1 \sqcup G_2$ is the disjoint union of the two DAGs: node set the disjoint union of the two node sets, edges exactly those of G_1 and of G_2 (*no edges between the parts*), attribute set $M_1 \sqcup M_2$. Its size (nodes, edges) is $|G_1| + |G_2|$ — *additive*.

The direct product is an operation on the *generated lattices* and it is the expensive, “unfolded” object. The coproduct is an operation on the *generators* and it is the cheap, “folded” object: you literally lay the two graphs side by side and change nothing. The content of this note is that, under closure decoding, the cheap folded thing generates the expensive unfolded thing.

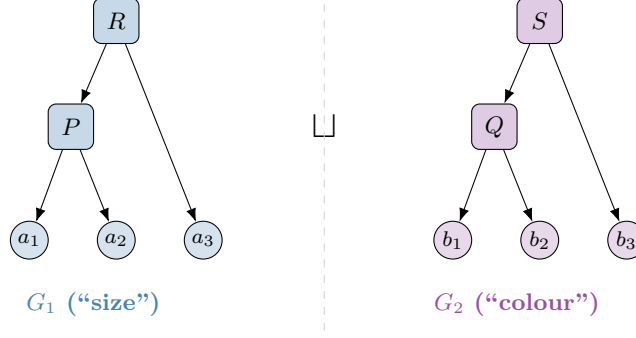


Figure 2: The coproduct $G_1 \sqcup G_2$ of two one-factor DAGs over *disjoint* attributes. It is nothing but the two graphs side by side — there are no edges across the dashed line. Four concept nodes total. By Theorem 1 this generates the 3×3 product lattice of Figure 3.

4 The result

The cleanest way to see *why* coproduct becomes product is to forget DAGs for a moment. The only property of closure decoding we use is that it is *separable*: the two parts write into disjoint attribute coordinates and their outputs are unioned.

Lemma 2 (Separable decoders factor). *Let $U = U_1 \sqcup U_2$ be a set of units and $M = M_1 \sqcup M_2$ a set of output attributes. Let $D : \{0, 1\}^U \rightarrow 2^M$ be any decoder that separates,*

$$D(s) = D_1(s|_{U_1}) \cup D_2(s|_{U_2}), \quad D_i : \{0, 1\}^{U_i} \rightarrow 2^{M_i},$$

with each D_i mapping into its own alphabet M_i . Write $\mathcal{I} = \text{image}(D)$ and $\mathcal{I}_i = \text{image}(D_i)$. Then the map $\varphi(A) = (A \cap M_1, A \cap M_2)$ is a bijection $\mathcal{I} \rightarrow \mathcal{I}_1 \times \mathcal{I}_2$, and it is an order isomorphism for $\subseteq$ (componentwise on the right). If each $\mathcal{I}_i$ is a lattice under $\subseteq$, then so is $\mathcal{I}$, and $\mathcal{I} \cong \mathcal{I}_1 \times \mathcal{I}_2$ as lattices.

Proof. Because the two decoders write into disjoint alphabets, every output splits as $D(s) = D_1(s|_{U_1}) \sqcup D_2(s|_{U_2})$ with the first part in M_1 and the second in M_2 . Hence for $A = D(s)$ we recover the two parts by intersection: $A \cap M_i = D_i(s|_{U_i})$, so $\varphi(A) = (D_1(s|_{U_1}), D_2(s|_{U_2})) \in \mathcal{I}_1 \times \mathcal{I}_2$. *Surjective:* any (A_1, A_2) with $A_i = D_i(s_i)$ equals $\varphi(D(s_1 \cup s_2))$, choosing the unit-vector that is s_1 on U_1 and s_2 on U_2 . *Injective:* $\varphi(A) = \varphi(A')$ forces $A \cap M_i = A' \cap M_i$ for both i , and since M_1, M_2 partition M , $A = A'$. *Order:* because the alphabets partition M , $A \subseteq A'$ holds iff $A \cap M_i \subseteq A' \cap M_i$ for both i — which is exactly the componentwise (product) order on the right. So φ and φ^{-1} both preserve order: an order isomorphism. An order isomorphism between finite lattices preserves all meets and joins, hence is a lattice isomorphism. $\square$

Closure decoding on a coproduct is a separable decoder: a node of $G_1 \sqcup G_2$ belongs to exactly one part, and (no cross edges) its closure lies entirely in that part's alphabet. So the theorem is just the instance.

Theorem 1 (Coproduct of generators = product of lattices). *Let G_1, G_2 be closure DAGs over disjoint attribute alphabets $M_1 \cap M_2 = \emptyset$. Then*

$$\mathcal{L}(G_1 \sqcup G_2) \cong \mathcal{L}(G_1) \times \mathcal{L}(G_2)$$

as lattices, via $A \mapsto (A \cap M_1, A \cap M_2)$. In particular the join and meet of the generated lattice are computed componentwise in the two factors.

Proof. Units are the nodes; $D = D_{G_1 \sqcup G_2}$ is separable with $D_i = D_{G_i}$ because a node's closure stays inside its own part (there are no edges across the coproduct). Its image is $\mathcal{L}(G_1 \sqcup G_2)$ and the parts' images are $\mathcal{L}(G_i)$, which are lattices by Lemma 1. Apply Lemma 2. $\square$

A worked example (Figure 2 $\rightarrow$ Figure 3). Take G_1 with $P = \{a_1, a_2\} \subset R = \{a_1, a_2, a_3\}$, so $\mathcal{L}(G_1) = \{\emptyset, \{a_1 a_2\}, \{a_1 a_2 a_3\}\}$, a 3-chain; and G_2 the same over b 's. The coproduct has the four concept nodes P, R, Q, S and six attributes. Its antichains pick at most one of the comparable pair $\{P, R\}$ and at most one of $\{Q, S\}$ — $3 \times 3 = 9$ antichains — decoding to the 9 attribute sets $\{(\text{one of } \emptyset, a_1 a_2, a_1 a_2 a_3) \cup (\text{one of } \emptyset, b_1 b_2, b_1 b_2 b_3)\}$. These are exactly the 9 pairs of $\mathcal{L}(G_1) \times \mathcal{L}(G_2)$, ordered as the grid of Figure 3. Four stored generators; nine represented concepts; and with k such chains, $2k$ generators represent 3^k concepts.

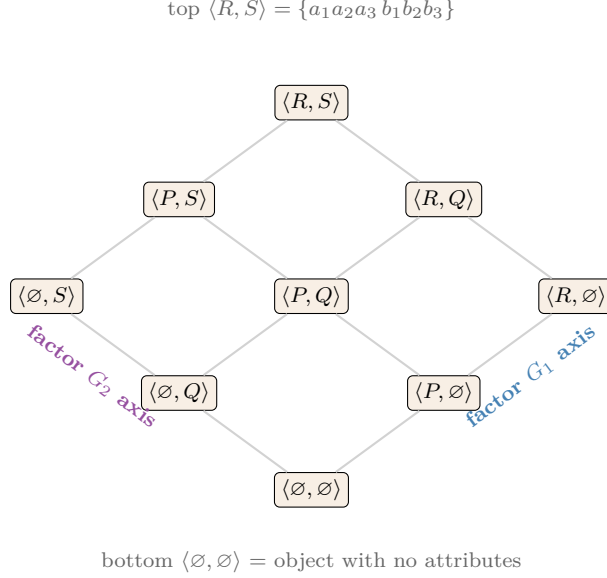


Figure 3: The lattice generated by the coproduct of Figure 2: a 3×3 grid, i.e. the direct product of two 3-chains. Each node is a *pair* $\langle \text{factor-1 state}, \text{factor-2 state} \rangle$; its attribute set is the union of the two states' attributes. The DAG stores only the two *axes* (four concept nodes); the nine grid points are generated, never stored. Moving up a north-east edge advances the G_1 coordinate, up a north-west edge the G_2 coordinate — the hallmark of a product order.

5 What the identity buys, and how it sits in known mathematics

Corollary 1 (Additive description, multiplicative expressiveness). $|\mathcal{L}(G_1 \sqcup G_2)| = |\mathcal{L}(G_1)| \cdot |\mathcal{L}(G_2)|$, while the generator $G_1 \sqcup G_2$ has $|G_1| + |G_2|$ nodes and edges. For k pairwise attribute-disjoint factors the represented lattice has $\prod_i |\mathcal{L}(G_i)|$ concepts and the generator has $\sum_i |G_i|$ parts. With k single-concept binary factors this is 2^k concepts from k generators.

This is the exponential gap between a factorial code and the concept lattice it stands for, written as an identity rather than a slogan. The generator is a *compressed* presentation of the lattice; the compression ratio is exactly the ratio of a sum to a product. Description-length-wise: the structure cost of storing the coproduct is additive in the factors, whereas materialising the product lattice as an explicit set of concepts is multiplicative — which is why any code that lands on the coproduct pays only the additive price. (This is a statement about the *structure* term alone; it holds for any monotone code over the generators, and needs no assumption about the data.)

Where this meets classical FCA. Two standard facts frame the result. First, apposition: if the same objects are described by two disjoint attribute blocks and one apposes the contexts, the concept lattice of the combined context is a *subdirect product* of the two block lattices —

it embeds in their direct product via extent intersection, and *equals* the full product exactly when every combination of block-extents is realised by some object [8]. Second, subdirect decomposition: by Birkhoff’s theorem every (finite) lattice is a subdirect product of subdirectly irreducibles [4, 5]. So “factor this concept lattice” *is* the subdirect-decomposition problem — genuinely nontrivial to run backwards on an already-built lattice.

Theorem 1 sits on the *generator* side of this picture and is correspondingly clean. Two points are worth keeping distinct:

- The lattice generated by the coproduct is the *full* direct product $\mathcal{L}(G_1) \times \mathcal{L}(G_2)$, because the DAG can switch on any combination of factor states regardless of what any data set contains.
- The lattice *realised by a particular data set* is the subdirect product of the classical result — the full product intersected with “which combinations actually occur.”

The gap between them is precisely which product cells are populated. A learner’s job is to install a coproduct generator whose full product *covers* the realised subdirect lattice, at additive cost — and, crucially, to do so *without* ever materialising the product and running Birkhoff backwards. Building the factors directly sidesteps the hard decomposition entirely.¹

6 The neural-code reading

FCA’s foundational equivalence is that a formal context (objects $\times$ attributes) and its concept lattice are two views of one thing, convertible both ways. A *neural code* in Barlow’s sense [1, 3] is the same data seen a third way: each object is a binary vector saying which *units* are active. In our model the units are the DAG’s nodes, the code for an object is the indicator of its activated antichain, and the decoder D_G is a fixed monotone Boolean map (an OR of closures) from code to attributes. Theorem 1 then reads, in code terms:

An additive architecture of units — two groups with no cross-wiring — under OR-of-closures decoding represents the multiplicative (product) space of concepts. Every combination of a factor-1 state and a factor-2 state is a representable object, expressed by co-activating one unit-group from each factor, not by a dedicated “combination” unit.

This is exactly Barlow’s *factorial code* [3, 2, 6, 7, 9]: independent factors of variation get independent groups of units; the combinatorial multitude of joint concepts is carried by *which units co-fire*, never by enumerating grandmother units for each combination. The exponential compression a factorial/sparse code delivers is the same fact as Corollary 1, now told about firing patterns instead of lattice nodes.

“Learn the code” vs. “learn the DAG” is not a dichotomy. A recurring temptation is to abandon the DAG and simply learn a sparse binary code as most of machine learning does, mapping to a DAG later — with the worry that mapping a learned code *to factored graphs* is itself hard. The model here dissolves the choice: the DAG *is* the decoder (the codebook) and the per-object antichain *is* the code; learning them is one act, not two. And the factorisation one would otherwise have to recover by hand falls out for free, because in a coproduct the factors are literally the *connected components* of the generator graph (Figure 2). Reading a learned

¹A terminological caution: FCA also has a *direct product of contexts* (object set $G_1 \times G_2$, a specific incidence) whose lattice is the *tensor* product, a different construction [8]. The operation relevant here is *apposition* of two attribute blocks over the *same* objects, whose lattice is a subdirect product of the direct product of the factor lattices. Theorem 1 is about the direct product of the *generated* lattices, on the generator side.

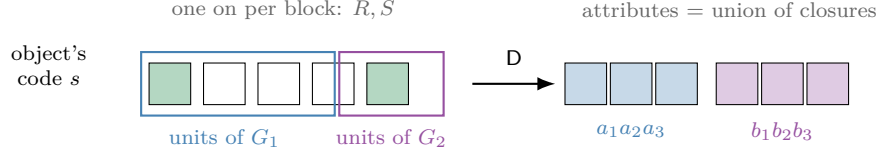


Figure 4: The same fact as a code. The unit vector splits into two blocks with no cross-wiring (additive architecture). Firing one unit per block and decoding by union of closures yields one cell $\langle R, S \rangle$ of the product lattice of Figure 3. All 3×3 cells are reachable this way; none needs a dedicated “ R -and- S ” unit.

coproduct’s factors is a linear-time graph traversal; reading a materialised product’s factors is subdirect decomposition. The lesson is to keep the representation *folded* (a coproduct of small generators) rather than unfold it into a product and try to refold — the same conclusion Barlow’s efficient-coding argument reaches from the redundancy side.

7 Boundary conditions worth remembering

The identity is clean but conditional. Three conditions matter, and it is worth being explicit about which are needed for the *representation* theorem and which only for a *learner* to prefer the factored form.

(B1) Disjoint alphabets — needed for the theorem. The whole proof rests on $M_1 \cap M_2 = \emptyset$ so that intersection recovers the two parts. If the two factors *share* an attribute, that attribute cannot be assigned to one part, φ is no longer a bijection, and $\mathcal{L}(G_1 \sqcup G_2)$ is only a *subdirect* product — the connected components no longer cleanly separate. Attribute-level entanglement between factors is exactly the “overlapping concepts” regime studied in the companion note, and it is where the product story stops being exact. So the product-lattice blow-up (independent blocks) is, perhaps counterintuitively, the *benign* case; the hard case is overlap *within* a block.

(B2) Statistical independence — needed for a learner to *prefer* the coproduct. The theorem says the coproduct *can represent* the product; it does not say a description-length learner will *choose* the coproduct over some partly-materialised product. That is a separate, data-dependent fact. Under per-use (usage) pricing, a dedicated cross-factor concept for a combination $\langle A, B \rangle$ earns coding bits only if that combination occurs *more often than independence predicts* — i.e. only if the factors are statistically dependent (positive pointwise mutual information). For independent factors the cross concept has zero information gain and cannot pay its usage rent, so the learner declines it and the coproduct is the optimum. For *dependent* factors, some cross concepts do pay, and the true optimum is a *partial* product — a few combination nodes grafted onto the two factors. Keeping (B1) and (B2) separate is important: (B1) is algebra, (B2) is statistics.

(B3) Realisability — which product cells the data populates. Even with disjoint, independent factors, a finite sample may not exhibit every one of the $|\mathcal{L}(G_1)| \cdot |\mathcal{L}(G_2)|$ combinations. The generator still *represents* them all (the theorem is about representability); the data merely exercises a subset. This is the gap between the full product (generator side) and the subdirect product (data side) of §5.

8 Empirical confirmation

We checked the prediction directly with the project’s MDL learner (whose scoring is described in the companion note). We planted two *independent* two-level hierarchies on disjoint 15-attribute blocks, concatenated them into one 30-attribute matrix over $N = 1000$ objects, and — to put the product-lattice pressure at maximum — arranged that *all* 36 combinations of an A -concept and a B -concept co-occur in the data. Classical FCA on this matrix would materialise (a large part of) the 6×6 product. The learner sees only the matrix.

quantity	value
co-occurring (A -concept, B -concept) pairs in data	36/36 (full product pressure)
concepts the learner returns	10
of which cross-block (attributes in both blocks)	0
learned concepts confined to block A / block B	5 / 5
objects whose code activates a concept from <i>both</i> factors	991/1000
worst per-factor planted-concept extent Jaccard	0.79 (A), 0.78 (B)

Table 1: Two independent planted factors, concatenated, handed to the MDL learner. It returns the two factors as *disconnected components* (five concepts each, zero cross-block), and represents the product in the object codes: almost every object co-activates one concept from each factor. The worst-case factor recovery (0.79) sits just under the project’s 0.80 match threshold — a small-block, finite-noise artefact — but the *structural* claim, zero cross-block concepts, is exact. This is Theorem 1 in the direction (B2) predicts: independent factors $\Rightarrow$ coproduct optimum.

The learned loading graph is literally two connected components, one per block; the product lives entirely in the antichain codes (Figure 4), never in the concept set. That is the theorem and the boundary condition (B2) behaving exactly as claimed. The experiment script is preserved with this note (see the project README) for re-running.

9 Relation to the project, and open directions

Within the `mdl-fca` project this note answers a design worry: that learning “one DAG” would force the exponential product lattice into a single tangled graph. It does not. With disjoint, independent feature groups the description-length optimum is the coproduct of the per-group DAGs, and the product is carried by the codes — so the ideal “product of factor DAGs” is what the learner already produces, as an emergent disconnected graph, with no factorisation penalty and no mutual-information term added by hand. But the note is written to stand alone: the core (Lemma 2, Theorem 1) is a fact about separable closure codes and factorial representations, true of any learner or none.

Directions worth pursuing later:

- **Make the factorisation a first-class output.** The learner returns one (disconnected) DAG; emitting its connected components as named factors is a linear-time post-step (§6). Worth doing so the model *reports* “this is $G_1 \sqcup G_2$ ” rather than leaving it implicit in the connectivity.
- **Partial products for dependent factors (B2).** When factors are not independent, characterise which cross concepts pay rent, and whether the learner recovers the minimal set of “bridge” concepts — the graded interpolation between full coproduct and full product.
- **Interaction with overlap (B1).** The companion note’s overlapping-concept limitation and this note’s disjoint-alphabet condition are the same boundary seen from two sides. A

usage code that lets a shared child pay rent (conditional/DAG-aware coding, or noisy-OR) is also what would let entangled factors be represented; unifying the two is the natural next theoretical step.

- **Beyond closure decoders.** Lemma 2 needs only separability, not closure. Any decoder that writes disjoint output blocks from disjoint unit blocks factors the same way; mapping which richer decoders (noisy-OR, thresholded linear) still yield clean products would extend the result past deterministic closure.

Summary

An additive combination of concept generators represents a multiplicative concept lattice. Precisely: over disjoint attribute alphabets, the coproduct of two closure DAGs generates the direct product of the lattices they generate individually (Theorem 1), because closure decoding is a separable decoder and separable decoders factor (Lemma 2). The generator grows as a sum while the represented lattice grows as a product — the exponential compression of a factorial code, stated as a lattice identity (Corollary 1) rather than an intuition. Read as a neural code, this says independent factors of variation deserve independent, non-interacting groups of units, with every joint concept carried by co-activation rather than a dedicated unit — Barlow’s factorial code — and it dissolves the apparent choice between learning a sparse code and learning a DAG, since the DAG is the decoder, the code is the activation, and the factorisation is just the generator’s connected components. The identity is exact under three conditions worth remembering: disjoint alphabets (algebra, needed for the theorem), statistical independence (statistics, needed for a compressor to *prefer* the factored form), and realisability (which product cells the data populates). A direct experiment confirms it: a description-length learner given two independent planted hierarchies returns the two factors and zero cross-factor concepts, representing all their combinations in the object codes.

References

- [1] Horace B. Barlow. Possible principles underlying the transformations of sensory messages. In Walter A. Rosenblith, editor, *Sensory Communication*, pages 217–234. MIT Press, 1961.
- [2] Horace B. Barlow. Single units and sensation: A neuron doctrine for perceptual psychology? *Perception*, 1(4):371–394, 1972.
- [3] Horace B. Barlow. Unsupervised learning. *Neural Computation*, 1(3):295–311, 1989.
- [4] Garrett Birkhoff. Subdirect unions in universal algebra. *Bulletin of the American Mathematical Society*, 50(10):764–768, 1944.
- [5] Brian A. Davey and Hilary A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002.
- [6] Peter Földiák. Forming sparse representations by local anti-Hebbian learning. *Biological Cybernetics*, 64(2):165–170, 1990.
- [7] Peter Földiák and Malcolm P. Young. Sparse coding in the primate cortex. In Michael A. Arbib, editor, *The Handbook of Brain Theory and Neural Networks*, pages 895–898. MIT Press, 1995.
- [8] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

- [9] Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, 1996.